SafeCall AI

Complete Backend Architecture, Native Classes & Real-Time Alert Pipeline Guide

System Name: SafeCall AI	**Target Environment:** Android Native (Java 17 / API 34)
Scoring Model: 4-Factor WeightedRuleEngine	**Database:** Embedded SQLite + Cloud Token Sync
Author / Project: SafeCall Engineering Team	**Release Version:** Production v2.0

1. Executive Architecture Overview

SafeCall AI is a dual-tier connected healthcare platform built specifically for elderly users living independently. Unlike generic cloud-dependent solutions, SafeCall AI executes its core telemetry processing and urgency classification **directly on the mobile handset** using native Android operating system facilities, an embedded SQLite database, and a deterministic Java rule engine. Sensitive call records and personal phone numbers never leave the user's phone for remote AI inference.

Core architectural pillars:

- **Zero ML Hallucination:** Urgency scores (0–100) are evaluated using a deterministic, 100% auditable mathematical rule engine rather than unpredictable deep learning models.
- **Zero-Latency Telephony Interception:** Intercepts native telephony broadcasts via Android OS TelephonyManager within milliseconds of an incoming call ring.
- **Multi-Gate Anomaly Pipeline:** Enforces a 3-gate filter (accumulated frequency, 60s grace period, and score severity) to virtually eliminate false alarms.
- **Dual-Mode Operation:** Full offline resilience with local SQLite queueing, seamlessly syncing with paired Caregivers via Render cloud backend and Firebase Cloud Messaging (FCM).

2. Device Pairing Protocol & Multi-Device Security

Device pairing links an elderly user's phone with one or more designated caregiver smartphones through a cryptographic, ephemeral token handshake. This prevents unauthorized surveillance and guarantees server-side consent.

Step	Actor	Protocol & Technical Operation
1. Code Generation	Senior Device	Senior taps *Generate 6-Character Pairing Code*. The phone issues a POST /api/pairing/generate request. Backend generates a cryptographically secure 6-character uppercase alphanumeric code (e.g. SWFTUB) with a strict 300-second (5 min) Time-To-Live (TTL).
2. Real-Time Countdown	Senior Device	The 6-char code is rendered in the UI with a live 1-second countdown timer. If already connected, the Senior can tap *Re-generate New Code* anytime to link an additional caregiver or reconnect.
3. Token Verification	Caregiver Device	Caregiver inputs the 6-character code under *Pair Senior* and taps *Connect*. Device sends POST /api/pairing/verify with the Caregiver's Bearer JWT.

4. Link Establishment	Backend Engine	Backend validates token, ensures code is unexpired, matches Senior ID to Caregiver ID, and creates an active relationship record. Returns confirmed pairing payload.
5. State Synchronization	Both Devices	Senior device's background polling loop detects the new link, plays a soft confirmation chime, and displays the Caregiver under *Active Vigilance* with an *Unlink* option.

3. Android Telephony Broadcast Interception & Finite State Machine (FSM)

Real-time incoming call detection is powered by SafeCallPhoneStateReceiver.java. The receiver models the call lifecycle as an explicit **Finite State Machine (FSM)** transitioning across Android OS telephony states: CALL_STATE_IDLE, CALL_STATE_RINGING, and CALL_STATE_OFFHOOK.

Current State	Trigger / Broadcast	Next State	Engine Action Executed
IDLE	EXTRA_STATE_RINGING	RINGING	Captures incoming phone number, normalizes E.164 format, records ringStartTime = currentTimeMillis().
RINGING	EXTRA_STATE_IDLE	IDLE (MISSED)	Duration = 0s. Flags call as **MISSED**. Dispatches call to SequenceTracker and WeightedRuleEngine in background thread.
RINGING	EXTRA_STATE_OFFHOOK	OFFHOOK (ANSWERED)	Senior picked up the call. Sets callAnsweredTime = currentTimeMillis(). Cancels any active grace-period timers.
OFFHOOK	EXTRA_STATE_IDLE	IDLE (TERMINATED)	Call ended. Computes talk duration: (now - callAnsweredTime) / 1000. Logs answered record with neutral urgency score.

4. Complete Breakdown of Core Android Backend Classes

The SafeCall AI Java codebase follows strict Object-Oriented principles, separating concerns across Telephony Receivers, Repositories (DAO), Evaluation Engines, Anomaly Trackers, and Native WebView Bridges.

Class Name & Package	Role, Responsibility & Architectural Pattern
SafeCallPhoneStateReceiver com.safecall.receiver	**BroadcastReceiver (Observer Pattern):** Intercepts android.intent.action.PHONE_STATE and NEW_OUTGOING_CALL. Contains the FSM logic to distinguish missed from answered calls. Sanitizes telephone strings and offloads processing to a dedicated background worker thread to prevent Android ANR (Application Not Responding) timeouts.
WeightedRuleEngine com.safecall.engine	**Pure Domain Service (Strategy Pattern):** Evaluates a CallContext object against the 4 context factors (Identity max 25, Frequency max 35, Timing max 10, Anomaly max 30). Produces an immutable UrgencyResult containing the score (0–100), urgency Level enum (LOW, MEDIUM, HIGH), and a human-readable explanation.

SequenceTracker com.safecall.engine	**Multi-Gate Anomaly Pipeline:** Implements the 3-gate non-response anomaly workflow. Accumulates missed call sequences, initiates a 60-second grace period timer upon 4 consecutive missed calls, and aborts if the elder answers, calls back, or interacts with the handset.
FamilyAlertManager com.safecall.engine	**Policy & Dispatch Coordinator:** Manages threshold filtering (standard 80/100), quiet-hours evaluation, and cooldown throttling (preventing repeated spam notifications within 10 minutes). Serializes verified alerts for FCM transmission or offline queueing.
CallRepository & CallDatabaseHelper com.safecall.data	**Data Access Layer (Repository Pattern):** Wraps an Android SQLite database containing three core tables: contacts, call_history, and pending_alerts. Guarantees ACID compliance, thread-safe access, and fast query execution through indexed telephone columns.
SafeCallNative com.safecall.bridge	**JavaScript Interface Bridge:** Exposes privileged native methods to the React frontend running inside Android WebView: getCallHistory(), getContacts(), saveContact(), saveAlertSettings(), launchDialler(), vibrate(), and triggerOfflineSync().
MainActivity com.safecall	**Host Application Entry Point:** Configures WebView security settings (disables unrestricted universal access, blocks mixed content, restricts bridge access to local assets), handles Android runtime permissions (READ_PHONE_STATE, READ_CALL_LOG), and manages app lifecycle.

5. The 4/4 Engine Check System & Mathematical Model

The SafeCall AI **4/4 Engine Check System** deterministically evaluates every incoming phone call against four independent, additive check gates implemented in WeightedRuleEngine.java. Every call executes all 4 checks in sequence without skipping: Check 1 (Caller Identity Gate, 25 pts), Check 2 (Frequency Window Gate, 35 pts), Check 3 (Timing Context Gate, 10 pts), and Check 4 (Historical Baseline Gate, 30 pts). The maximum sum is strictly bounded: 25 + 35 + 10 + 30 = 100 points.

Urgency Score Formula:
Score = Min(100, W_Identity + W_Frequency + W_Timing + W_Anomaly)

Factor & Weight	Trigger Condition in Java Rule Engine	Allocated Pts	Rationale & Clinical Justification
1. Caller Identity (Max 25 pts)	• Priority == EMERGENCY • Priority == TRUSTED • Priority == NORMAL / UNKNOWN	**+25 pts** **+15 pts** +0 pts	Designated doctors, hospitals, and primary family contacts carry elevated baseline urgency over unfamiliar numbers.
2. Call Frequency (Max 35 pts)	• 4+ calls in 15m window • 3 calls in 15m window • 2 calls in 15m window • 1 call in 15m window	**+35 pts** **+22 pts** **+10 pts** +0 pts	Rapid, repeated redials within a narrow 15-minute window strongly indicate an urgent situation requiring immediate attention.

3. Timing & Context (Max 10 pts)	• Late-Night (23:00–05:00) with repeated or emergency caller • Late-Night standalone • Daytime (06:00–22:59)	**+10 pts** **+5 pts** +0 pts	Middle-of-the-night calls from family members or doctors are statistically rare and warrant instant escalation.
4. Historical Anomaly (Max 30 pts)	• ≥50% unanswered rate (valid ≥ 3) • ≥33% unanswered rate (valid ≥ 3) • Insufficient history / normal	**+30 pts** **+15 pts** +0 pts	Compares against the contact's historical baseline. When normally answered callers go unanswered, it indicates a potential senior incapacitation.

6. How Real-Time & Offline Alerts Are Dispatched

Alert dispatch is fully automated. When a real call or high-risk simulation event triggers an urgency score ≥ 80/100, the alert travels through the dual-mode transmission pipeline:

A. Online Real-Time Cloud Pipeline:

1. **Payload Construction:** The Android engine builds a JSON payload containing idempotentId, callerName, callerPhone, score, level: HIGH, explanation, and timestamp.
2. **Cloud Transmission:** Dispatches a secure POST /api/alerts/trigger request to the Render backend.
3. **Caregiver Notification:** The backend authenticates the sender, verifies the active paired relationship, and broadcasts an FCM Push Notification to the Caregiver's handset. If the Caregiver dashboard is active, it sounds a soft warning chime and presents an interactive Alert Banner.
4. **Acknowledgment:** Caregiver taps *Acknowledge Alert*, issuing POST /api/alerts/acknowledge, updating the audit trail across both devices.

B. Resilient Offline SQLite Queuing Pipeline:

If the Senior's device has no internet connectivity (airplane mode, rural signal loss), SafeCall AI ensures **zero alert loss**:
1. The alert is saved to the SQLite table pending_alerts with status QUEUED.
2. An in-app warning badge indicates: ■■ *N offline alert(s) queued*.
3. The native service registers an Android ConnectivityManager.NetworkCallback. As soon as cellular data or Wi-Fi is restored, triggerOfflineSync() flushes all queued records to the cloud backend with idempotent deduplication.

7. Comprehensive Viva Voce & Oral Defense Reference

Viva Examiner Question	Definitive Architectural Answer
Q1: Why did you choose a Weighted Rule Engine over Deep Learning / LLMs?	In emergency elderly healthcare, AI black-boxes are hazardous due to non-deterministic hallucinations, latency, high battery drain, and lack of legal auditability. The 4-factor additive model calculates urgency deterministically in <1 ms on low-end hardware with 100% explainable justifications for emergency responders.
Q2: How do you prevent false alarms from waking caregivers at night?	Through a 3-gate filter in SequenceTracker.java: Gate 1 requires 4+ consecutive missed calls in 15 mins. Gate 2 enforces a 60-second grace period (cancelling if senior answers or dials out). Gate 3 checks whether severity score ≥ 80/100 and verifies against caregiver quiet-hour schedules.
Q3: How are user call privacy and GDPR / HIPAA compliance handled?	SafeCall AI does NOT record, stream, or transcribe audio conversations. Telemetry evaluation is performed solely on call metadata (timestamps, caller identity, frequency). All call logs are stored in on-device SQLite, not in third-party clouds.
Q4: What happens if the caregiver's phone is unreachable?	The cloud backend queues the push notification via Firebase Cloud Messaging (FCM) High-Priority channel, which triggers persistent heads-up notifications that bypass Android Doze mode and ringtone mutes.